A Reinforcement Learning Architecture for Spatio-Temporal Reallocation in Smart Container Terminals: Independent Proposal and Acceptance Policies with Counterfactual Cost Learning

Geon-U Kim and Bong-Jun Choi✉

Department of Computer Engineering, Dongseo University, Busan, Republic of Korea
bjchoi@gdsu.dongseo.ac.kr

Abstract. Container terminals assign each external-truck job to a yard block and an appointment time before the truck arrives. A fixed assignment, however, cannot reflect congestion that emerges later. This paper proposes a reinforcement learning architecture that jointly revisits the block and arrival time of an announced job during operation. Two independent policies decide whether to propose a reallocation and whether the destination block or time slot should accept it. A central procedure then commits only mutually accepted candidates that satisfy resource constraints. Moving one job affects not only that job but also the waiting time of later trucks, the crane work sequence, and vessel supply. To capture these delayed effects, each policy learns from cost differences between discrete-event simulation branches that start from the same state and differ in one action. Online decisions use only the trained networks. We evaluate the architecture in a continuous synthetic environment whose demand scale and time-of-day variation are based on ranges reported in the literature. Compared with no reallocation, the proposed architecture reduced total operating cost by 14.7%. It also cost less than a rule-based policy with the same spatio-temporal action range on 20 of 28 days ($p = 0.036$). Arrival-time adjustments accounted for 91.3% of the reduction, and the four most congested days accounted for 90.5%. These results suggest that spatio-temporal reallocation is most useful as a selective response to observed congestion rather than as a continuous intervention.

Keywords: Smart port, Container terminal, Truck appointment system, Yard block reallocation, Counterfactual cost learning, Reinforcement learning

1 Introduction

Container throughput continues to grow, but terminal land and equipment cannot expand at the same rate. The resulting pressure appears as congestion in yard and gate operations. Smart-port research therefore addresses not only automated equipment but also operational decisions. Studies of port efficiency show that

automation, intelligence, and environmental design affect performance through different channels. Actual gains thus depend on how a terminal turns observed information into operating decisions [1]. Existing work follows two main directions. On the temporal side, truck appointment systems spread arrival peaks and allow terminals and carriers to coordinate arrival times while retaining their own costs [2, 3]. On the spatial side, yard-operations research improves container placement, work sequencing, and crane-interference management [4–7]. Recent studies have also applied reinforcement learning to real-time yard-crane dispatching [8].

Most of these approaches assume that the working block and appointment time have already been fixed before the truck arrives. They improve operations within that assignment. During operation, however, demand may concentrate in particular time slots or blocks, while crane queues also change. An assignment that was reasonable when announced may then become inefficient. This paper revisits the assignment itself. For an announced job, it jointly decides *where* the job should be handled and *when* the truck should arrive. Moving one job affects later truck queues, the crane work sequence, and vessel supply. An immediate movement cost cannot capture these effects. Difference rewards and multi-agent counterfactual baselines address this problem by comparing one action with an alternative [9, 10]. The destination must also decide whether it can absorb the job, separately from any benefit to the proposing side. We therefore use independent proposal and acceptance policies, while a central procedure checks mutual consent and feasibility.

Fig. 1 summarises the full workflow. During operation, observation, proposal, acceptance, commitment, and execution form a 60-s decision cycle. Offline, the observed action and two alternatives are compared from the same state and random stream to construct targets for the proposal and acceptance cost networks. The expensive counterfactual branches are therefore absent from online operation. Sect. 3 gives the policy, commitment, and learning details.

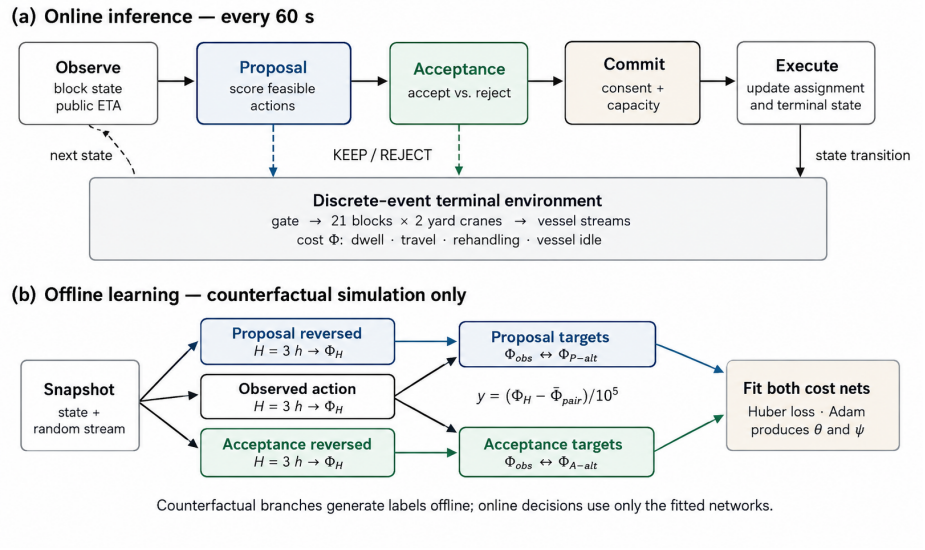


Fig. 1: Overall concept of the proposed system. (a) Online operation proposes a reallocation from the observed state, asks the destination to accept it, and commits only changes that satisfy mutual consent and capacity. (b) Offline learning compares the observed action with counterfactual alternatives over 3 h to fit the two cost networks. Online operation uses only the fitted networks.

This study asks four questions. Can the two independent policies support spatio-temporal reallocation? Can expensive counterfactual learning remain separate from online inference? Does the architecture reduce operating cost? Finally, can the effects of the action range and of learning be distinguished? Our contributions are an *independent policy structure*, a *spatio-temporal action range*, *counterfactual cost learning*, a *separation of learning and operation*, and a *decomposable evaluation* of execution, action range, and learning effect. Sect. 2 reviews the relevant literature. Sect. 3 presents the architecture and learning signal, Sect. 4 describes the experimental environment, and Sect. 5 reports the results.

2 Related Work

2.1 Smart Ports and Terminal Operations

Yen et al. measure port efficiency with data envelopment analysis and then use Tobit regression to relate the scores to smart-port design attributes. This separates *how efficient a port is* from *what explains the difference*. Their results also treat automation, intelligence, and environmental design as distinct channels [1]. This distinction motivates attention to the decision layer in addition to the equipment layer.

On the temporal side, Chen et al. manage truck arrivals with time windows to relieve gate congestion. Their method reshapes the arrival profile instead of

adding capacity [2]. Phan and Kim model a decentralised setting in which trucking companies and the terminal coordinate appointments while retaining their own costs [3]. This setting motivates our decision to model acceptance separately rather than absorb it into one objective. Kourounioti and Polydoropoulou use aggregate terminal data to model truck arrivals by time of day and show that arrivals are far from uniform [11]. The value of shifting an arrival therefore depends on both its original and target positions on the arrival curve. The environment in Sect. 4 reflects this time-of-day variation.

On the spatial side, Ng and Mak study yard crane scheduling in container terminals and sequence crane jobs to reduce truck waiting [4]. Speer and Fischer extend this problem to automated systems with several cranes that can interfere within the same block [5]. Petering and Murty use long-run simulation to study how block length and crane deployment affect terminal-wide performance [6]. We adopt their continuous, multi-week design with state carried forward. Schwientek et al. compare dispatching methods in simulation and show that their ranking depends on terminal conditions [7]. We therefore hold the dispatching rule fixed in the main experiment and examine its sensitivity separately.

2.2 Reinforcement Learning as Optimisation over Future Cost

Conventional dispatching rules often rank actions by their immediate cost. In our problem, however, the effect of moving a job appears later in truck queues, the crane work sequence, and vessel supply. Immediate movement cost alone cannot capture these effects. Reinforcement learning can instead compare actions by the cost that follows over a horizon. Wang et al. apply a PPO-based method to real-time yard-crane scheduling across multiple blocks [8]. Their method optimises work sequencing in the execution layer. We address the preceding assignment layer, where the block and arrival time are selected.

This perspective leads to two modelling choices. First, the networks do not output action probabilities. Each network maps the observed state and one candidate action to a predicted cost score, and the policy selects the candidate with the lowest score. The score is not an absolute operating cost. Labels are centred within each counterfactual pair (Sect. 3.4), so a network predicts what one action costs *relative to* the action it was compared against. Because the same constant is subtracted from both actions of a pair, the ordering of candidates is unchanged. The method therefore approximates action values rather than a policy, as deep Q-networks do [12], but it forms its target differently: the target is a simulated cost difference measured to the end of a fixed horizon, not a bootstrapped estimate that reuses the network's own output. Second, we use small multilayer perceptrons. Feedforward networks are general function approximators [13], but the input structure also supports this choice. Each input is a fixed-length numeric vector, and candidates are scored independently. The input has no temporal order for a recurrent or attention model to exploit, and the number of labelled decisions is too small to support a much larger model. One set of weights is also shared across all blocks. A small perceptron is therefore an appropriate first model for this setting.

When several parties act, the horizon cost is joint and the effect of one action is difficult to isolate. Wolpert and Tumer construct difference rewards that remain aligned with the global objective while responding to an individual action [9]. Foerster et al. apply the same idea to deep multi-agent learning, where a centralised critic marginalises one agent's action to form a counterfactual baseline [10]. Both obtain the future term from a learned value estimate. We use the same decomposition principle but measure the future term with simulation. Starting from the same state, we rerun the discrete-event simulator after changing one action. The regression target is therefore the monetary cost difference accumulated over a fixed horizon, not a bootstrapped advantage. This choice adds offline computation but keeps the learning signal in the same units as the operating objective.

Kim et al. study learning-based resource control under uncertainty in building demand response. They combine measured environmental records with synthetic parameters for unit-level variation and identify the synthetic component explicitly [14]. Sect. 4.1 follows the same principle by separating literature-based ranges from our design values.

3 Proposed Architecture

Fig. 2 separates the architecture into the operational path, the terminal environment, and the offline learning path. The operational path reviews each newly announced job once and changes its assignment through independent proposal and acceptance policies and a central commitment procedure. The terminal environment then measures truck dwell, additional travel, rehandling, and vessel idle cost. The learning path clones the state and random stream immediately before a decision and compares the observed action with counterfactual alternatives. This computation remains outside the operational path.

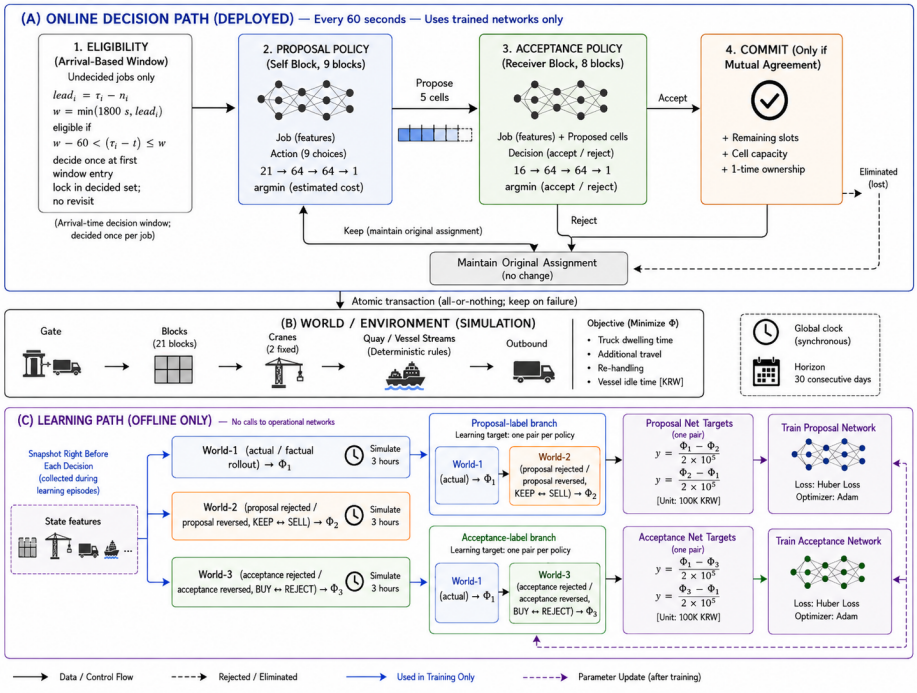


Fig. 2: Detailed flow of the proposed architecture. (a) Every 60 s, the operational path checks eligibility, proposal, acceptance, and commitment; failure of mutual consent or capacity keeps the original assignment. (b) The discrete-event environment links the gate, 21 blocks, two yard cranes per block, and vessel operations to calculate cost Φ. (c) Offline learning compares up to three worlds from the same state and random stream over 3 h and constructs separate labels for the proposal and acceptance cost networks.

3.1 Notation, Actions, and Policies

The decision interval is $\delta = 60\,\text{s}$. The current assignment of job i is $z_i = (b_i, \tau_i)$, where b_i is the block and τ_i is the announced arrival time. A reallocation action is $a_i = (b'_i, \Delta_i) \in \mathcal{A}_i(t)$ and gives $z_i(a_i) = (b'_i, \tau_i + \Delta_i)$. Keeping the original assignment is $a_i^0 = (b_i, 0)$. Block reallocation has $b'_i \neq b_i$, and arrival-time adjustment has $\Delta_i > 0$; a candidate may change both. Outbound jobs are excluded from block reallocation because the retrieved container has a fixed location.

Let n_i be the announcement time, g_i the gate-in time, and r_i an indicator of prior review. A job becomes eligible not when it is announced but when its arrival comes within a review window $w_i = \min(W,\ \tau_i - n_i)$ of the present, with $W = 1800\,\text{s}$, so that the decision is taken on the freshest state the lead time allows:

$$\mathcal{E}_t = \{i \mid w_i - \delta < \tau_i - t \le w_i,\ g_i > t,\ r_i = 0\}. \tag{1}$$

Each job therefore enters exactly one decision interval and is never revisited. For a job announced at least W ahead of its arrival the decision falls exactly

W before that arrival—about 46 minutes later than the median announcement, which is where the freshness comes from—while a job announced with a shorter lead is reviewed as soon as it is announced. A job announced no earlier than its own arrival has $w_i = 0$ and is never eligible; such jobs are 14.1% of the demand under the lead-time distribution of Sect. 4.3, and the architecture leaves their assignment untouched. The proposal policy uses the current block state and job-action features o_i^P. It scores each candidate and selects the one with the lowest predicted cost score (Sect. 3.3):

$$\hat{a}_i = \arg\min_{a \in \mathcal{A}_i(t)} \hat{c}_\theta^P(o_i^P, a), \qquad p_i = \mathbf{1}[\hat{a}_i \neq a_i^0]. \tag{2}$$

If the policy proposes a change, the destination block or target time slot becomes the accepting party. The acceptance policy uses the proposal message and destination state o_i^A to compare acceptance with rejection:

$$q_i = \mathbf{1}\left[\hat{c}_\psi^A(o_i^A, \hat{a}_i, \text{accept}) \leq \hat{c}_\psi^A(o_i^A, \hat{a}_i, \text{reject})\right]. \tag{3}$$

If the target resource has no remaining capacity, the request is rejected before the network is called. The mutually accepted candidates form $\mathcal{J}_t = \{i \in \mathcal{E}_t \mid p_i q_i = 1\}$.

3.2 Central Commitment Constraints

The central procedure does not create new candidates. It considers only $\mathcal{J}_t$, sorted by the acceptance policy's predicted cost, with lexicographic tie-breaking. Let $x_i \in \{0, 1\}$ indicate commitment, $F_b(t)$ denote the free slots in block b, and $K_k(t)$ denote the remaining quota of time slot k. The procedure enforces

$$\begin{aligned} x_i \leq p_i q_i, \qquad & \sum_{i \in \mathcal{J}_t} x_i \mathbf{1}[b_i' = b] \leq F_b(t), \\ \sum_{i \in \mathcal{J}_t} x_i \mathbf{1}[k(i) = k] \leq K_k(t), \qquad & \sum_{t \in \mathcal{T}} \mathbf{1}[i \in \mathcal{E}_t] \leq 1. \end{aligned} \tag{4}$$

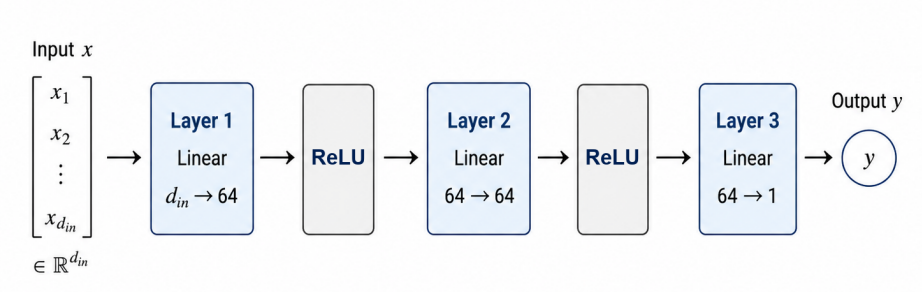
Block capacity is enforced in every experiment. When time-slot quota data are unavailable, we set $K_k(t) = \infty$. Results under this setting represent the potential range of arrival-time adjustment. All committed assignments are applied together at the end of the decision epoch. An earlier commitment therefore cannot change the state observed by another candidate in the same epoch. The structure guarantees two properties: only mutually accepted candidates are committed, and each job is reviewed once.

3.3 Policy Models and Inputs

The functions $\hat{c}_\theta^P$ and $\hat{c}_\psi^A$ in Sect. 3.1 are cost networks. Neither produces action probabilities. Each maps an observed state and one candidate to a scalar cost score. The score is pair-centred rather than absolute: the two actions of a

counterfactual pair are centred on their mean before training (Sect. 3.4), so the network predicts what a candidate costs relative to the action it was rolled out against. Mean subtraction removes a constant that is common to both actions, so the ranking of candidates, and therefore the arg min in (1) and the comparison in (2), is unaffected. The policy evaluates every candidate with the same network and selects the arg min of the learned score. Both networks are multilayer perceptrons with weights shared across blocks. The proposal network has a 21-dimensional input, and the acceptance network has a 16-dimensional input. Both use two 64-unit hidden layers with ReLU. They contain 5,633 and 5,313 parameters, respectively, or fewer than 11,000 in total. Inputs include current waiting, inventory, remaining crane work, the public expected arrival time, route difference, and announced volume near the target time. Unrealised arrival and completion times and simultaneous responses from other accepting parties are excluded.

Fig. 3 shows one cost network. A candidate enters as a fixed-length vector $x \in \mathbb{R}^{d_{in}}$. It passes through three linear layers, with ReLU after the first two, and produces one scalar: the pair-centred cost score of that candidate. Because the output is one value per candidate, the candidate-list length may change across decision epochs without changing the model. The same weights also apply to every block. The model therefore learns a common rule for evaluating a candidate in the current block state, rather than a separate rule for each block. The policies differ only in input dimension: 21 for proposal and 16 for acceptance.



$$\mathrm{MLP}(x) = W_3\ \mathrm{ReLU}\big(W_2\ \mathrm{ReLU}(W_1 x + b_1) + b_2\big) + b_3$$

Fig. 3: Structure of one cost network. The input is a fixed-length feature vector for one candidate, and ReLU follows each of the two 64-unit hidden layers. The output is one pair-centred cost score. A decision evaluates every candidate with the same network and takes the arg min. The proposal and acceptance policies share this structure and differ only in input dimension, 21 and 16 respectively.

3.4 Counterfactual Cost Labels

For each sampled decision, we clone the state and random stream just before branching. We then run up to three worlds for three hours:

$$\begin{aligned} \phi_i^F &= \Phi_H(\text{observed}), \qquad \phi_i^P = \Phi_H(\text{proposal reversed}), \\ \phi_i^A &= \Phi_H(\text{acceptance reversed}), \end{aligned} \tag{5}$$

with $H = 10{,}800\,\text{s}$. The alternative world always reverses what the policy actually did. If the proposal policy proposed a change, the alternative keeps the current assignment; if it kept the assignment, the alternative commits the cheapest candidate under the current weights. The acceptance alternative reverses acceptance and rejection. A decision without an acceptance request produces only the proposal comparison.

For each policy, we centre the two actions of the pair on their mean, $\bar{\phi}_i^P = (\phi_i^F + \phi_i^P)/2$, and divide by KRW 100,000. This gives $y_{i,F}^P = (\phi_i^F - \bar{\phi}_i^P)/10^5$ and $y_{i,P}^P = (\phi_i^P - \bar{\phi}_i^P)/10^5$, that is, $\pm$ half the pairwise cost difference. We apply the same centring to (ϕ_i^F, ϕ_i^A) for the acceptance policy. The reference point is therefore the pair itself rather than a fixed action: the target states how much better or worse one action was than the single alternative that was rolled out against it in the same world. Mean subtraction removes the cost the two worlds share without changing the order of the two actions.

3.5 Training Settings and Operational Separation

We sample at most 64 decisions per day. Exploration decreases linearly from 0.50 to 0.05, and decisions are split 80/20 into training and validation sets. We optimise with Adam (3×10^{-4}) and Huber loss ($\beta = 1.0$). A few counterfactual differences are much larger than the rest, so squared error would allow these samples to dominate the fit [17]. Minibatches contain at most 256 rows. The number of updates is $\max(50, \min(600, 4 \times \text{rows}))$, and gradients are clipped at 1.0. Counterfactual branches run in parallel across CPU cores, and each day's labels are replaced after the daily update. During evaluation, weights are frozen, exploration is disabled, and no counterfactual branch is called.

The labels are therefore not a fixed dataset. Every counterfactual pair branches from the action the current policy actually took, so the states that are labelled, and the alternative each is compared against, both move as the weights change; the labels of one pass are discarded after that pass. In this respect the scheme is on-policy Monte-Carlo advantage estimation with a simulator-generated baseline: the target is a complete simulated rollout to the horizon, and the method uses neither bootstrapping nor a policy gradient.

Training both networks for 24 passes took 4.7 CPU hours. Online inference requires one forward pass through a small network for each candidate. This cost is small relative to the accompanying constraint check. The main computational expense is counterfactual label generation, which is performed offline and never enters the operating path.

4 Experimental Environment

4.1 A Literature-Based Synthetic Environment

We evaluate the architecture in a literature-based synthetic environment, not in the operating record of a particular terminal. The studies in Sect. 2 provide the modelling ranges: 20 blocks [7], multiple cranes per block [5], continuous multi-week operation [6], strong time-of-day variation in truck arrivals [11], three vessel classes with different call volumes [7], and a reported range of crane handling times [6]. Within these ranges, we use 21 blocks, two cranes per block, 30 continuous days with state carried across day boundaries, three vessel classes, and a reference crane service time of 180 s. Sect. 4.3 defines the arrival curve. These settings are design choices. The citations show that the values lie within ranges used in prior models; they do not imply calibration to a specific port.

4.2 Terminal and Time Settings

The yard contains 21 blocks and 42 yard cranes, with two cranes per block. Each block has 1,440 slots and starts at 65% occupancy. The gate and quay lie at opposite ends of a one-dimensional layout. The decision interval is 60 s. Operations continue for 30 days, and the middle 28 days are compared. Cranes give priority to service jobs and then to the job with the shortest expected processing time. A day boundary separates cost accounting; it does not reset the terminal. Inventory, queues, crane states, and unfinished jobs carry over. The first and last days connect these continuous boundaries.

4.3 External Truck Demand and the Arrival Process

We construct external-truck demand from ranges reported in prior work. The literature supports similar demand scales and strong time-of-day variation [11]; the specific curve below is our synthetic design. Daily demand is drawn from 3,500, 5,000, 7,500, 12,500, and 15,000 trucks with probabilities 0.30, 0.30, 0.25, 0.10, and 0.05 (Fig. 4a). The first three levels represent smooth, normal, and high demand. The last two represent congested and heavily congested conditions. Within each day, job announcement times follow an arrival density $f(t)$. It combines a uniform base containing 38% of the volume with three Gaussian components containing the remaining 62% (Fig. 4b):

$$f(t) = 0.38\,U(0,24) + 0.62 \sum_{m=1}^{3} w_m \mathcal{N}(t; \mu_m, \sigma_m^2), \tag{6}$$

where (μ_m, σ_m, w_m) equals $(10, 1.5, .317)$, $(15, 2.5, .633)$, and $(21, 1, .05)$, with time in hours. The uniform base keeps nighttime arrivals above zero. The state carried across a day boundary is therefore not created by an artificially empty night. The peak locations, widths, and weights are design values for comparing

congestion sensitivity across demand levels, not estimates from a particular terminal. **Training and evaluation use the same curve shape**; only demand level and random draw differ. We therefore do not evaluate transfer to an arrival profile with a different shape.

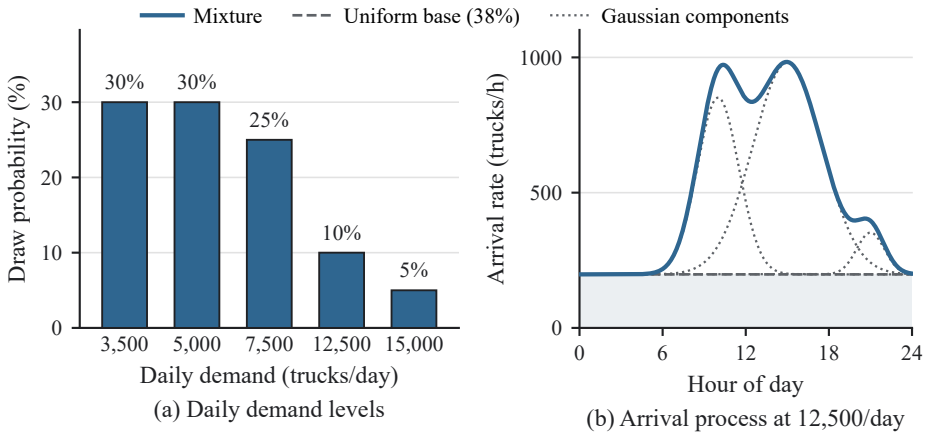


Fig. 4: External truck job demand. (a) The five daily demand levels and the probability with which each is drawn, independently for each day. (b) The time-of-day arrival process at 12,500 trucks per day, with the uniform base and the three Gaussian components drawn separately beneath the mixture they sum to. The literature supports the demand magnitudes and the fact that arrivals are far from uniform over the day, whereas the shape of the mixture is a construction of this study.

4.4 Vessels and Quay Operations

We use three vessel classes. Table 1 summarises their size, assigned quay cranes, idle cost, and daily call mix.

Table 1: Vessel classes and daily call mix. Shares use the expected 3.3 vessel calls per day. The 0.3 call for large vessels is the expectation from adding one large vessel with probability 0.30.

Class	GT	TEU	STS (cranes)	Idle cost (KRW 1,000/h)	Calls/day (mean share)
Small	50,000	3,000	2	149.5	2.0 (60.6%)
Medium	100,000	7,500	4	299.0	1.0 (30.3%)
Large	150,000	14,000	6	448.5	0.3 (9.1%)

For a vessel with capacity C TEU, call volume is $C \cdot u$, where $u \sim U[0.15, 0.30]$. One quay crane processes 27.5 moves/h, within the 24, 30, and 36 moves/h values used in the literature [7]. We sum blocking across the assigned quay cranes and divide by their number to obtain idle time per vessel.

On the yard side, the reference service time is 180 s per move, giving a nominal capacity of $\mu_{YC} = 21 \times 2 \times 3600/180 = 840$ moves per hour. A move is a container movement, not a truck job. One truck job may require several moves and rehandles, so we report the two units separately.

4.5 Cost Function

The total cost over the interval $[d, d+1)$ is the sum of four terms:

$$\Phi_d = C_{wait,d} + C_{move,d} + C_{rehandle,d} + C_{vessel,d}. \tag{7}$$

Truck dwell costs KRW 40,000 per hour, with the same rate applied a second time beyond one hour. The monetary anchor comes from the safe trucking freight rate, while the time conversion is a design choice [15]. Additional crane movement costs KRW 60,000 per working hour, and rehandling costs KRW 2,000 per event. Both are design values. Vessel idling costs KRW 2.99 per GT-hour, converted from the excess berthing rate of KRW 29.9 per 10 GT-hour [16]. Let h_i be the dwell time of truck i, m the additional crane working time, R the number of rehandles, and T_v the policy-related idle time of vessel v. The terms are

$$C_{wait} = \sum_i 40{,}000\{h_i + \max(0, h_i - 1)\}, \quad C_{move} = 60{,}000\, m, \tag{8}$$

$$C_{rehandle} = 2{,}000\, R, \quad C_{vessel} = \sum_v 2.99\, GT_v T_v. \tag{9}$$

Structural vessel time that the policy cannot change, such as berthing and quarantine, is recorded separately as a diagnostic. All four cost terms are differences between cumulative values measured at the same day boundary.

The 30-day training run reveals two properties of this environment. First, waiting dominates total cost. C_{wait} accounts for 96.4%, compared with 2.2% for rehandling, 1.2% for vessel idling, and 0.1% for additional crane movement. Conclusions based on Φ are therefore driven mainly by the regulated safe-freight-rate coefficient [15]. The two design-based coefficients have little influence. Second, waiting and service levels deteriorate sharply as demand rises. Waiting represents 83.2% of cost at 3,500 trucks per day, 91.7% at 7,500, and 99.1% at 15,000. Across the month, mean truck turn time is 0.89 h and the 90th percentile is 2.01 h. Of 206,904 truck jobs, 5.0% remain unfinished on the day of announcement. More importantly, the daily 90th percentile ranges from 0.79 h on the lightest days to 7.49 h on the heaviest, a 9.5-fold difference. Congested days differ not only in cost level but also in cost composition. The reallocation policy is expected to exploit this change.

5 Numerical Experiments

5.1 Scope of the Results

The results come from a 30-day continuous diagnostic run with random seed 9,900,950, which was not used in training. We compare the middle 28 days and

use the first and last days only to connect the continuous boundaries. During evaluation, weights are frozen and exploration is set to zero. All policies share the same truck, vessel, and service random streams on a given day. We record the four cost terms, demand level, transaction types, and unfinished jobs for each day. The source code, raw policy results, and figure scripts are released with the paper.

5.2 Confirmation of Architectural Operation

Implementation checks, commitment constraints, and automated tests confirm four structural properties. Proposal and acceptance use separate models. One-sided consent never produces a commitment. Realised future arrival and completion times are excluded from policy inputs. State also continues across day boundaries. The run logs confirm two additional properties. Frozen-policy evaluation made **zero** counterfactual simulation calls. The identity check also passed at all four sampled points with counterfactual labels: a branch that changed no action reproduced the action and cost of the main run.

5.3 Cost Reduction and Paired Daily Comparison

Table 2 compares each policy's 28-day total cost with no reallocation. A positive value means a cost reduction, whereas a negative value means a cost increase. The rows are grouped by research question rather than ranked in one list.

Table 2: Total cost reduction by policy. Each reduction is measured against the 28-day total cost without reallocation. Positive values mean lower cost, whereas negative values mean higher cost.

Policy	Action range	Reduction vs. no reallocation
Proposed policy	block + time	**+14.72%**
Proposed, time only	time	+13.44%
Proposed, block only	block	−1.31%
Untrained model	block + time	+7.97%
Rule: least-loaded slot	time	+3.41%
Rule: least-loaded block + slot	block + time	+1.51%
Rule: least slack	block	+0.53%
Rule: first-come-first-served	block	+0.17%
Rule: shortest processing time	block	+0.05%
Rule: net gain	block	−0.48%
No reallocation	—	0.00%

Over 28 days, total cost was KRW 10.00 bn without reallocation and KRW 8.52 bn with the proposed policy. The reduction was KRW 1.471 bn, or 14.7%. Time adjustment alone reduces cost by 13.44%, whereas block reallocation alone raises cost by 1.31%. The total reduction therefore comes mainly from arrival-time adjustment.

We run every policy with the same random streams to reduce simulation noise, following Kleijnen's common-random-number method [18]. Days are treated as independent comparison units. We report two paired tests because they answer different questions. A two-sided sign test counts only the number of days on which one policy is cheaper, so it is blind to how large each day's difference is. A Wilcoxon signed-rank test ranks the differences by magnitude before summing them, so a policy that wins on fewer days but by a wider margin is scored differently by the two. The distinction matters here because the proposed policy acts selectively: it is not designed to win on every day but to reduce cost sharply on congested ones. With 28 days and no tied differences, both tests are computed exactly. Table 3 reports both.

Table 3: Paired daily comparison between the proposed policy and each comparison policy. "Proposed lower" counts the days on which the proposed policy costs less. Both p values are two-sided and exact: a sign test, which uses only the direction of each daily difference, and a Wilcoxon signed-rank test, which also uses its magnitude. Values below 0.05 are in bold.

Purpose	Comparison policy	Proposed lower days	Sign test p	Signed-rank p
Learning effect	Untrained model	18/28	0.185	0.164
Same action range	Least-loaded block + slot rule	20/28	**0.036**	**0.015**
Block rule	Least slack	20/28	**0.036**	**<0.001**
Block rule	First-come-first-served	22/28	**0.004**	**<0.001**
Block rule	Shortest processing time	21/28	**0.013**	**0.006**
Block rule	Net gain	21/28	**0.013**	**0.003**
Baseline	No reallocation	26/28	**<0.001**	**<0.001**

The proposed policy costs less than the rule with the same block-and-time action range on 20 of 28 days (sign test $p = 0.036$, signed-rank $p = 0.015$). It costs less than the untrained model on 18 days, but this difference is not statistically clear under either test ($p = 0.185$ and $p = 0.164$). At the 5% level the two tests agree on every row of Table 3. They are not equally sensitive, however: the signed-rank p is the smaller of the two in six of the seven rows, which is what a concentrated effect produces. The architecture's action range and the additional effect of learning must therefore be interpreted separately. The proposed policy is more expensive on only two days, and both differences are small, whereas the largest reductions all occur on congested days (Fig. 5).

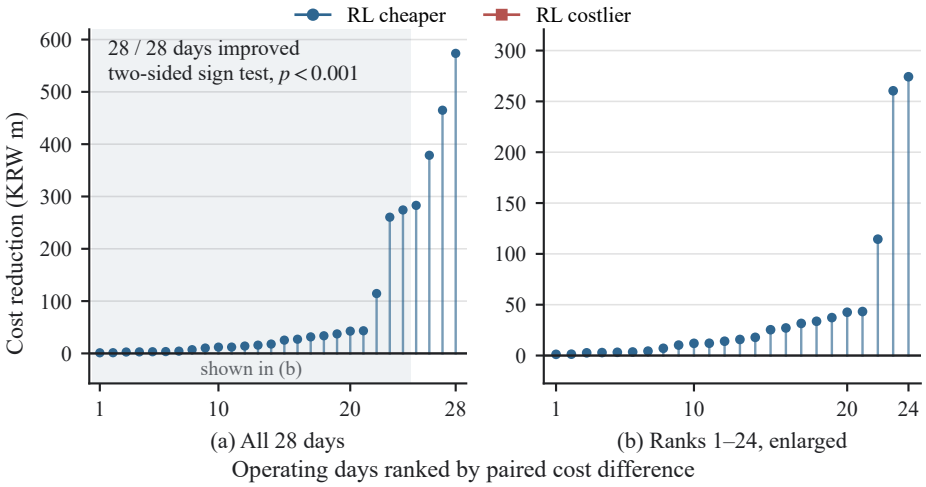


Fig. 5: Paired daily cost difference against no reallocation over the same 28 days with the same random streams. Days are sorted by the difference, and a positive value means the proposed policy costs less. (a) All 28 days; four congested days dominate the axis, so (b) enlarges the shaded ranks 1–24. The proposed policy costs less on 26 of the 28 days (two-sided sign test, $p < 0.001$). The two exceptions are small, KRW −17.1 m and KRW −0.5 m, whereas each of the four congested days exceeds KRW 290 m.

5.4 Demand Level and the Decomposition of Actions

Table 4 decomposes the reduction by demand level. The four days with 12,500 or 15,000 trucks account for 90.5% of the total. This concentration supports developing the architecture toward a policy that *intervenes selectively when congestion is observed* rather than continuously. Restricting the policy to one action type at a time separates the spatial and temporal contributions. Arrival-time adjustment accounts for 91.3% of the total reduction, mostly on the four congested days (Fig. 6). Block-only reallocation saves KRW 0.124 bn under smooth and normal demand but raises cost at the highest demand, producing a net increase of KRW 0.131 bn. The policy should therefore adjust its use of spatial and temporal actions by demand level. Time-slot quotas were open in this diagnostic run. The arrival-time results thus describe the *potential range* of time shifting, not an executable schedule. Sensitivity analysis with finite quotas can define the practical range.

Table 4: Cost reduction by demand level (KRW bn, relative to no reallocation). Columns three to five separate the allowed actions. The last column compares the trained and untrained models. Arrival-time adjustment and learning contribute mainly on the four congested days, whereas block reallocation raises cost at the highest demand.

Daily demand	Days	Both actions	Time only	Block only	Due to training
3,500	12	0.047	−0.013	0.057	0.056
5,000	8	0.060	0.029	0.067	0.029
7,500	4	0.032	0.008	0.000	−0.049
12,500	2	0.611	0.573	−0.014	0.144
15,000	2	0.722	0.747	−0.241	0.495
Total	**28**	**1.471**	**1.344**	**−0.131**	**0.675**
share of total		100%	**91.3%**	—	**45.9%**

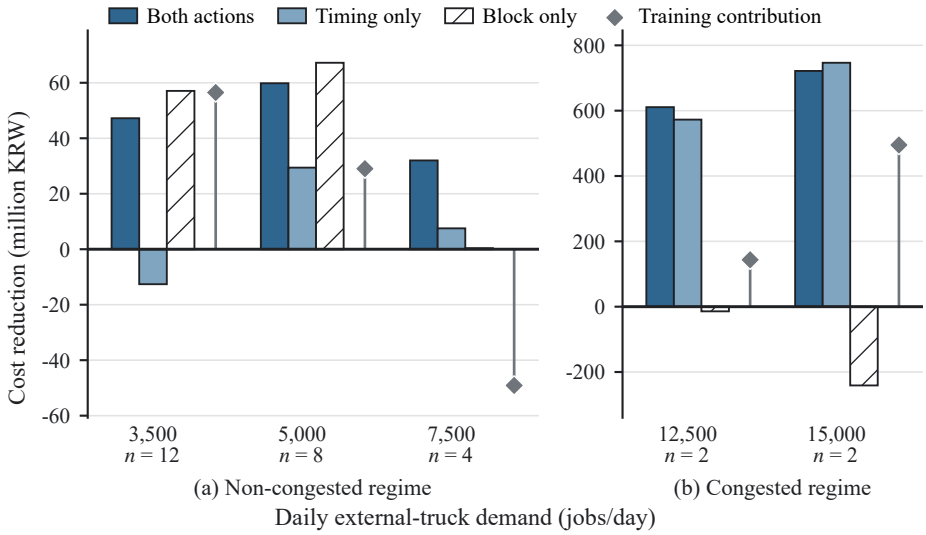


Fig. 6: Cost reduction by demand level, decomposed by action range. The values are those of Table 4, in million KRW (1 bn = 1,000 m). The vertical axis is split by demand because the two congested levels are more than ten times larger than the rest: (a) 3,500–7,500 and (b) 12,500–15,000 jobs per day, with the number of days observed printed as n under each tick. Bars measure a restricted policy against no reallocation. The diamond is *not* a fourth action: it is the trained model minus the untrained one—the training contribution, that is, the last column of Table 4—and carries a different mark for that reason. Moving from the non-congested to the congested regime, arrival-time adjustment and the training contribution both grow sharply, whereas block-only reallocation turns negative.

5.5 Rule-Based Policies of the Same Action Range

To determine whether the difference in Sect. 5.4 comes from the action itself, the rule-based policies must receive the same action range. We therefore transfer the *least-loaded* criterion from the spatial axis to the temporal axis and add two rules. This change introduces no new parameter and uses the same congestion trigger as the other rules. Three findings follow. First, arrival-time adjustment under a rule reduces cost by 3.41% relative to no reallocation ($p = 0.0125$). The temporal action therefore has value by itself. Second, with the full spatio-temporal action range matched, the proposed policy costs less than the rule on 20 of 28 days ($p = 0.036$). Third, for arrival-time adjustment alone, the rule costs less on more days; the proposed policy is lower on only 8 of 28 days. Yet the proposed policy's total reduction is about four times larger. The rule is slightly better under smooth demand, whereas the proposed policy reduces cost 3.8–5.6 times more on congested days. Thus, the learned policy tends to produce large savings on expensive days rather than frequent small wins. Both the number of wins and the total reduction are needed to show this pattern. The rule using both actions also weakens under congestion. This suggests that the congestion sensitivity of block reallocation may be a property of the action, not only of one policy.

5.6 Learning Effect and Dispatching Sensitivity

We first examine model fit. Across 30 training iterations, the proposal network's Huber loss decreases from 0.079 to 0.029 on training decisions and from 0.268 to 0.119 on validation decisions. The acceptance network's loss decreases from 0.171 to 0.027 and from 0.315 to 0.136, respectively. These values are means over the first and last five iterations, with targets in units of KRW 100,000. The decline in validation loss indicates that the fit extends to unseen decisions. However, final validation loss remains about four times training loss. With about 56 labelled decisions per iteration, this gap supports the choice of a small model. As exploration decreases, the share of decisions with exactly zero counterfactual difference falls from 51.4% to 37.6%. Training runs 4,671 counterfactual worlds in total. Fig. 7 shows both loss curves together with the exploration schedule that accompanies them.

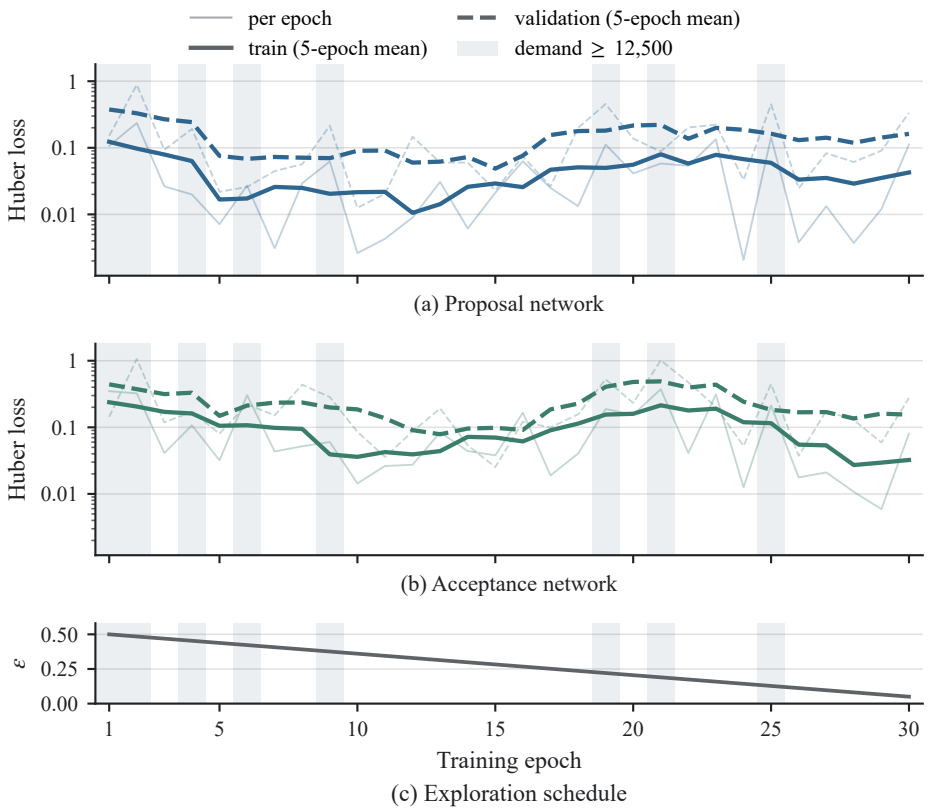


Fig. 7: Training and validation loss over 30 iterations, with the per-iteration value drawn faintly beneath a five-iteration mean. (a) The proposal network and (b) the acceptance network are drawn separately because they are fitted on different decisions, and the vertical axis is logarithmic because the loss spans three orders of magnitude. (c) The exploration probability ε decays linearly from 0.50 to 0.05. Shading marks the iterations whose demand level was drawn at 12,500 trucks per day or above. Because the demand level is redrawn every iteration, much of the visible fluctuation, including the rise around iterations 17–24, follows that draw rather than the fit.

Cost is KRW 10.00 bn without reallocation, KRW 9.20 bn with the untrained model, and KRW 8.52 bn with the trained model. Training therefore adds KRW 0.675 bn, or 45.9% of the total reduction. However, the trained model costs less on only 18 of 28 days (sign test $p = 0.185$, signed-rank $p = 0.164$). The mean daily difference is KRW -24.1 m, about 18 times the median of -1.3 m. This gap shows that the learning effect is concentrated rather than evenly distributed. The four congested days account for 94.6% of the learning effect, and the trained model costs less on all four. The corresponding counts at lower demand levels are 9/12, 5/8, and 0/4. For the remaining 24 days, the mean difference is -1.5 m and the bootstrap interval includes zero.

A stratified bootstrap, which resamples the daily differences within demand strata with 20,000 resamples, gives a 95% percentile interval of $[-32, -17]$ m per day for the mean difference. Stratification preserves the demand mix fixed by the design. Without it, the interval widens to $[-52, -3]$ m, and 99.3% of resamples remain negative. Both intervals exclude zero, but the stratified interval is partly narrow by construction because each congested stratum contains only two days. The interpretation is therefore limited. Learning produces a large total reduction and is lower on all four congested days, but the day-level sign test is not significant. Establishing the learning effect at a conventional significance level requires more congested days, not simply more days overall.

The ranking of crane-dispatching rules also changes with demand. The shortest processing time rule is 22–25% more expensive than the other rules under smooth and normal demand, but it is the least expensive from 7,500 trucks upward. It also has the lowest overall cost under the designed demand mix. The effect of reallocation should therefore be tested under other dispatching rules.

6 Conclusion

This paper proposed a reinforcement learning architecture that jointly revisits the yard block and arrival time of an announced truck job during operation. The architecture connects distributed appointment negotiation [3] and yard-crane operations [4, 5] in one spatio-temporal assignment problem. Reallocation affects later truck, crane, and vessel operations. Each policy therefore learns from a counterfactual cost difference measured over a fixed horizon, rather than from immediate movement cost. This is how multi-agent counterfactual learning [9, 10] enters the operating decision. Expensive branch simulation is used only during offline training. Online operation requires only small cost networks and constraint checks.

In the literature-based synthetic environment, the architecture carries spatio-temporal reallocation from observation through proposal, acceptance, commitment, and execution. Cost reductions arise mainly from arrival-time adjustments on congested days. This result supports the view that real-time smart-port information can guide assignment actions, not only monitoring [1]. The proposed policy also has lower total cost than a rule with the same action range. For arrival-time adjustment alone, however, the rule is lower on more days, while the proposed policy leads in total because of larger savings on congested days. Both daily win counts and total savings are therefore needed for interpretation.

6.1 Limitations

Seven limitations bound these results. First, the environment is synthetic. Its scale and variation follow reported ranges, but it is not fitted to the operating record of a specific terminal. The demand mix, arrival-curve coefficients, and two of the four cost coefficients are also design values. Second, training and evaluation use the same arrival-curve shape and differ only in demand level and

random draw. We therefore do not show transfer to an unseen arrival profile. Third, only four of the 28 days are congested, and most of the reduction occurs on those days. Confirming the learning effect requires more congested days, not simply more days overall. Fourth, we do not isolate the contribution of the independent acceptance policy. We confirm that one-sided consent never produces a commitment, but we do not measure the cost of removing the acceptance veto. Separate proposal and acceptance thus remain a design choice supported by operational checks rather than an ablation. Fifth, time-slot quotas are open, so the arrival-time results give a potential upper range rather than an executable schedule. Sixth, the three-hour counterfactual horizon is a design value. It combines a 30-minute review window, up to 120 minutes of deferral, and a 30-minute arrival-pressure margin. Finally, the main comparison uses one crane dispatching rule. Because rule rankings change with demand, another rule may change the measured size of the reallocation effect.

These limits can be tested one axis at a time. Actual appointment quotas would define the executable range of time adjustment. A wider set of congested conditions would clarify the contribution relative to the untrained model. This study organises observation, proposal, acceptance, commitment, execution, and counterfactual learning into one reproducible structure. As automated equipment and appointment information expand, the architecture can help connect real-time congestion information to explainable resource-assignment decisions.

Acknowledgments. This work was supported by Public-Private Joint Technology Commercialization R&D (Technology Transfer and Commercialization) (RS-2026-25529952) funded by the Ministry of SMEs and Startups (MSS, Korea).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Yen, B.T.H., et al.: How smart port design influences port efficiency: a DEA-Tobit approach. Res. Transp. Bus. Manag. **46**, 100862 (2023)
2. Chen, G., Govindan, K., Yang, Z.: Managing truck arrivals with time windows to alleviate gate congestion at container terminals. Int. J. Prod. Econ. **141**(1), 179–188 (2013)
3. Phan, M.-H., Kim, K.H.: Collaborative truck scheduling and appointments for trucking companies and container terminals. Transp. Res. B **86**, 37–50 (2016)
4. Ng, W.C., Mak, K.L.: Yard crane scheduling in port container terminals. Appl. Math. Model. **29**(3), 263–276 (2005)
5. Speer, U., Fischer, K.: Scheduling of different automated yard crane systems at container terminals. Transp. Sci. **51**(1), 305–324 (2017)
6. Petering, M.E.H., Murty, K.G.: Effect of block length and yard crane deployment systems on overall performance at a seaport container transshipment terminal. Comput. Oper. Res. **36**(5), 1711–1725 (2009)
7. Schwientek, A., Lange, A.-K., Jahn, C.: Simulation-based analysis of dispatching methods on seaport container terminals. ARGESIM Report **59**, 357–364 (2020)

8. Wang, W., et al.: Yard crane real-time scheduling among multi-block at terminal: a reinforcement learning based PPO approach. Transp. Res. E **207**, 104635 (2026)
9. Wolpert, D.H., Tumer, K.: Optimal payoff functions for members of collectives. Adv. Complex Syst. **4**(2–3), 265–280 (2001)
10. Foerster, J., et al.: Counterfactual multi-agent policy gradients. In: AAAI, vol. 32 (2018)
11. Kourounioti, I., Polydoropoulou, A.: Application of aggregate container terminal data for the development of time-of-day models predicting truck arrivals. EJTIR **18**(1) (2018)
12. Mnih, V., et al.: Human-level control through deep reinforcement learning. Nature **518**(7540), 529–533 (2015)
13. Hornik, K.: Approximation capabilities of multilayer feedforward networks. Neural Netw. **4**(2), 251–257 (1991)
14. Kim, S., Mowakeaa, R., Kim, S.-J., Lim, H.: Building energy management for demand response using kernel lifelong learning. IEEE Access **8**, 82131–82141 (2020)
15. Ministry of Land, Infrastructure and Transport: Notice on Safe Freight Rates for Cargo Vehicles Applicable in 2026. MOLIT Notice No. 2026-402 (2026)
16. Ministry of Oceans and Fisheries: Regulations on the Use of Port Facilities and Fees at Trade Ports, etc. MOF Notice No. 2025-218 (2025)
17. Huber, P.J.: Robust estimation of a location parameter. Ann. Math. Statist. **35**(1), 73–101 (1964)
18. Kleijnen, J.P.C.: Analyzing simulation experiments with common random numbers. Manag. Sci. **34**(1), 65–74 (1988)